

Experimento 2 - Calculadora de 4 Bits

1. Identificação do Grupo

Integrante 1: Douglas Monteiro Almeida Souza

NUSP 1: 10748048

Integrante 2: Guilherme Junji Tutiya

NUSP 2: 14576065

Integrante 3: Henrique Maruiti

NUSP 3: 12610243

<https://github.com/Douglas-At/PCS3732-laboratorio-processadores>

2. Especificação de Requisitos e Testes

Especificação de requisitos funcionais (RF) e **não-funcionais** (RNF, vide norma [ISO25010](#)) que o circuito a ser projetado deve atender e os **testes** associados a cada requisito mapeado, conforme especificado na Tabela 1.

Requisito	RF ou RNF	Teste	Resultado Esperado	Resultado Obtido	Evidências de Resultados
Breve descrição do requisito	Especificar se é requisito funcional ou não-funcional	Breve descrição do caso de teste	Descrever o resultado esperado	Para cada resultado apresentado, discutir se ele está adequado ou não.	Citar capturas de tela, fotos e outras evidências que podem ser inseridas após
Controlar LED's pela placa ESP	Não funcional	Verificar se conseguimos configurar as portas do arduino	Os LED's devem alternar a cada intervalo de 1 segundo	Os LED's alternaram entre aceso e apagado a cada 1 segundo	Os LED's acendem e desligam alternadamente. Vídeo
Teste da calculadora	Funcional	Verificar se a calculadora está funcionando corretamente	Os LED's devem realizar as equações dadas	Os LED's tiveram o comportamento esperado	Imagens

Tabela 1 - Requisitos e Testes Planejados

3. Arquitetura do Sistema

A arquitetura do sistema desenvolvida nesta aula utiliza o ESP32 como servidor web e ponto de acesso Wi-Fi (Access Point), permitindo que dispositivos se conectem diretamente à rede criada pelo microcontrolador, sem necessidade de acesso à Internet. Após conectar-se à rede do ESP32, o usuário acessa, por meio de um navegador web, uma interface de calculadora de 4 bits hospedada no próprio dispositivo. Nessa interface, são inseridos os operandos e a operação desejada, enquanto o processamento aritmético é realizado internamente pelo ESP32. Em seguida, o resultado da operação é enviado de volta ao navegador e exibido ao usuário em tempo real.

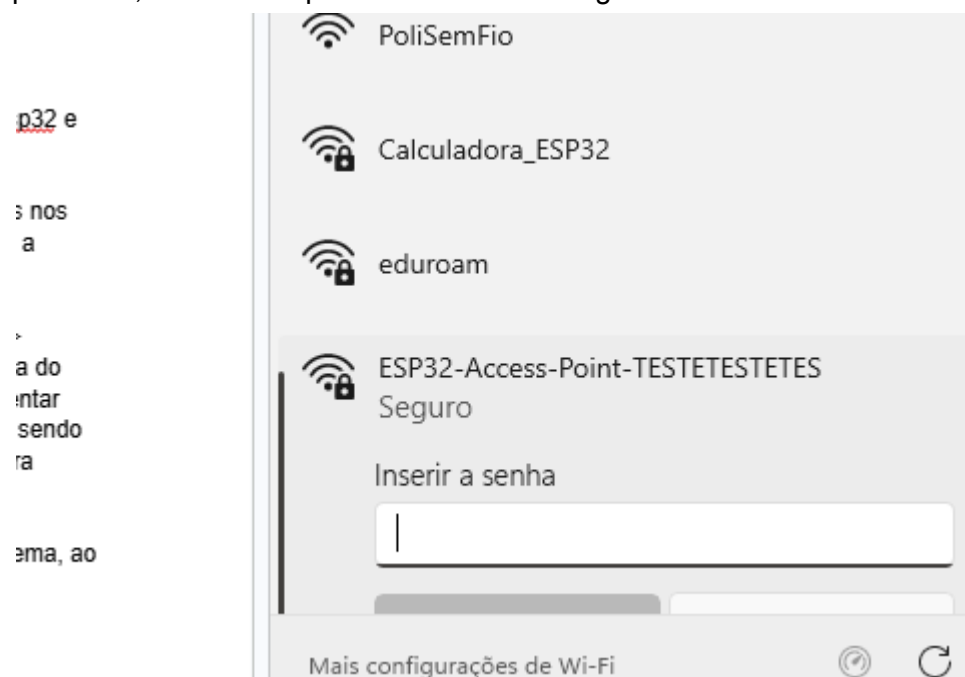
4. Método Experimental

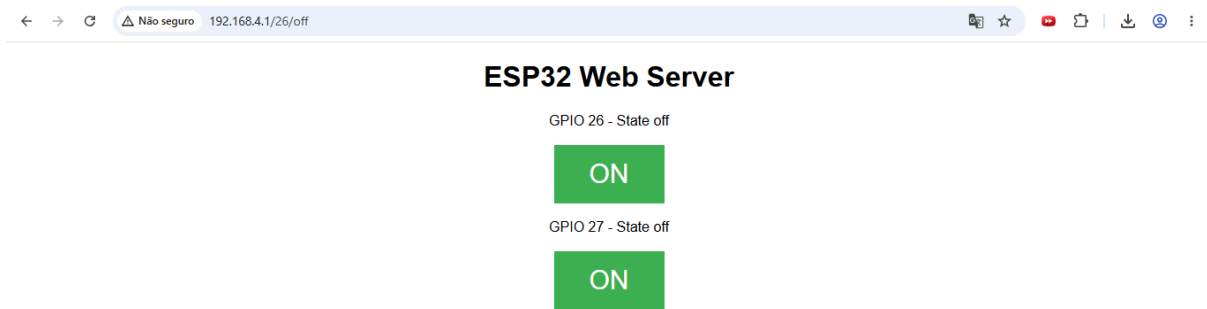
Testamos inicialmente se os leds estavam funcionando apenas pelo gnd e vcc da esp32 e todos os 4 leds estão operando normalmente,

Após isso testamos com uma adaptação do blink para fazer os 4 leds q posicionamos nos pins 4,5,6,7 para piscar juntos e apagar juntos e deu certo segue o video que mostra a evidencia do funcionamento  controle_4_leds.mp4

Agora para testar a parte de access point como indicado, usamos dos samples>wifi> access point e infelizmente nao apareceu a conexão, pegamos o codigo da referencia do site, <https://randomnerdtutorials.com/esp32-access-point-ap-web-server/> porem ao tentar compilar não foi nada, estavamos com dúvida se era o esp a entrada, se não estava sendo compilado chegamos até a testar um caso de fazer o esp piscar um led para ver seera compilado, trocamos o esp.

No final o erro era a entrada USB do computador da bancada b2, ele esta com problema, ao usar um powerbank como energia funcionou normalmente.





tela do teste do site indicado no slide

Apos essa parte experimental fomos para aplicação da calculadora usando binário de -8 a 7

```
int a = pegarParametro(req, "a").toInt();
int b = pegarParametro(req, "b").toInt();
String op = pegarParametro(req, "op");

// garante operandos dentro de 4 bits
a &= 0x0F;
b &= 0x0F;

int completo = 0; // resultado matematico real
bool overflow = false; // estourou os 4 bits?

if (op == "add") {
  completo = a + b;
} else if (op == "sub") {
  completo = a - b;
}

// se nao couber em 4 bits (0..15), marca overflow
if (completo < 0 || completo > 15) {
  overflow = true;
}

int resultado4bits = completo & 0x0F; // só os 4 bits baixos
mostrarNosLeds(resultado4bits);
```

amostra do codigo para mostra que a conta nao foi feita no javascript do html e sim no esp

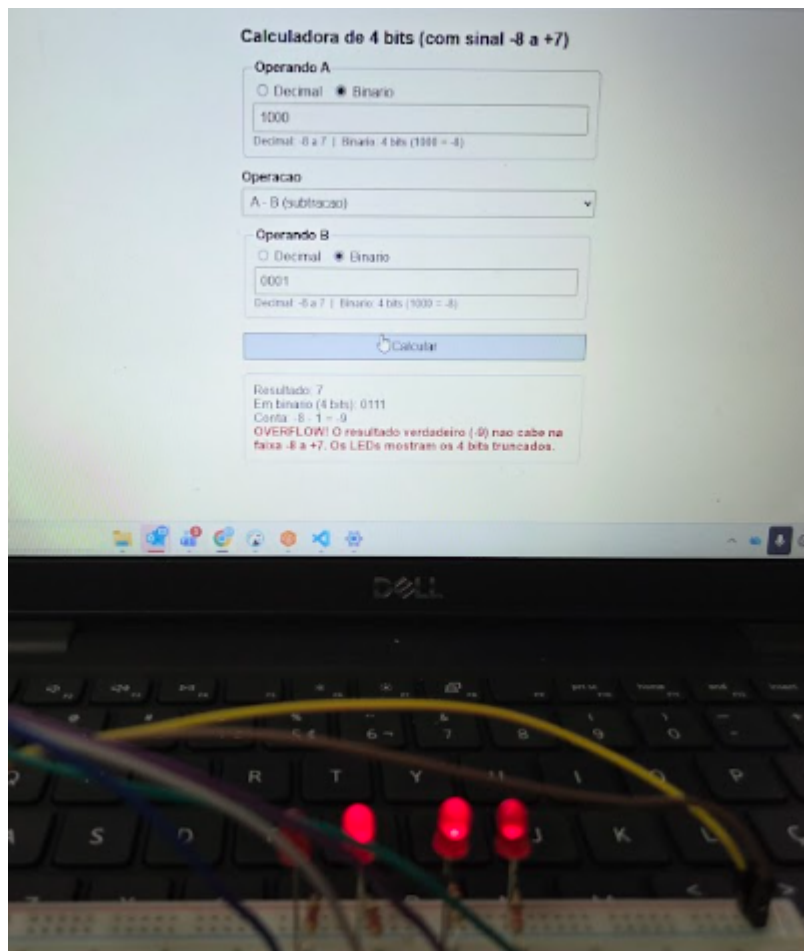


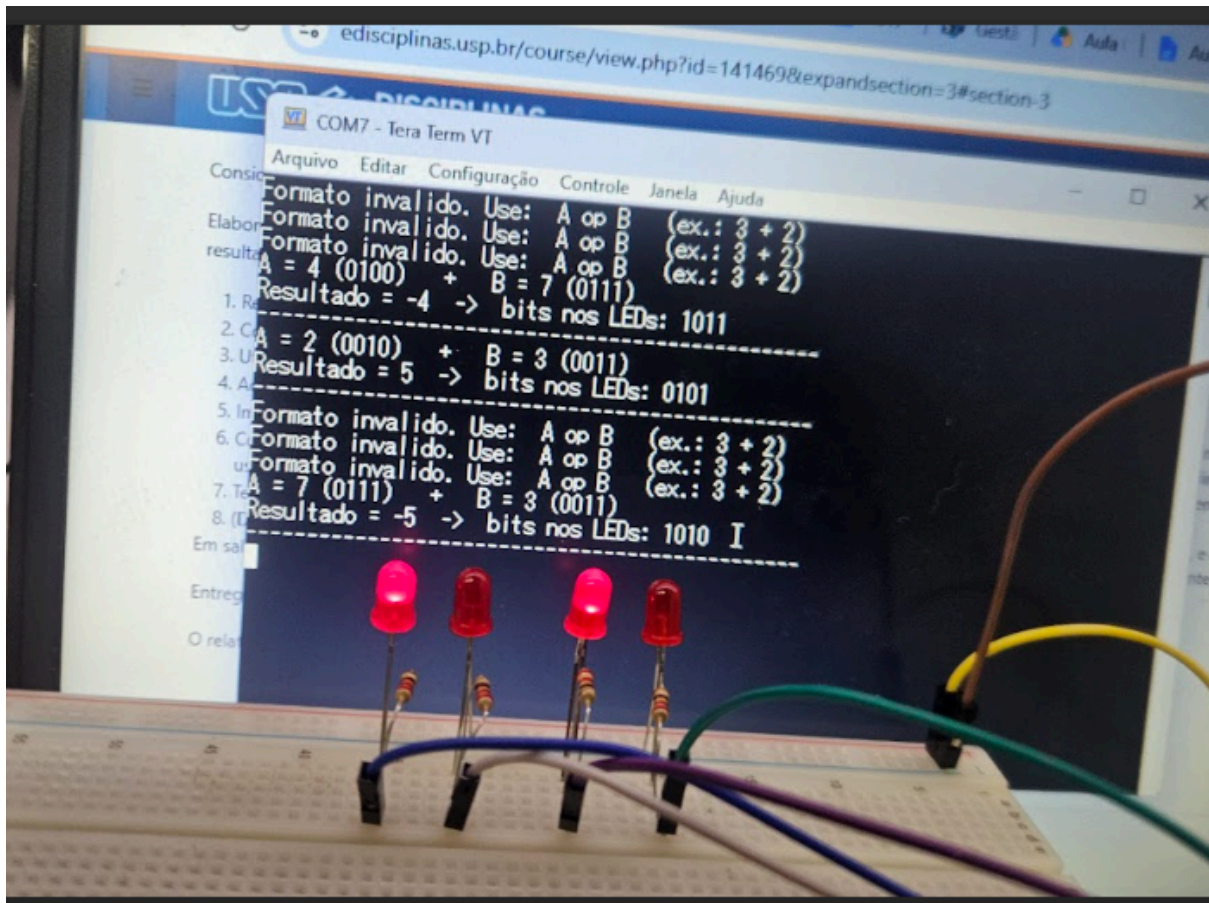
foto do exemplo da calculadora_overflow.ino (todos os codigos usando estão no github)

e por fim para completar o desafio precisamos fazer a conexão via serial, foi instalado o terraterm computador para poder usar o teclado como input

```
void loop() {
  if (Serial.available()) {
    String linha = Serial.readString();
    linha.trim();
    if (linha.length() == 0) return;

    // descobre o operador. Procuramos + ou - que seja o operador (nao o sinal do 1o numero).
    char op = 0;
    int posOp = -1;
    // começa a busca a partir do indice 1 para nao pegar um sinal negativo inicial
    for (unsigned int i = 1; i < linha.length(); i++) {
      char c = linha[i];
      if (c == '+' || c == '-') {
        // so e operador se vier depois de um espaco ou de um digito/letra (e nao logo apos outro sinal)
        char ant = linha[i - 1];
```

fazemos a leitura do serial e o resto do processo é idêntico, para a adaptação desse primeiro código para o de serial usamos o claude como auxílio do desenvolvimento. e chegamos no resultado da seguinte foto



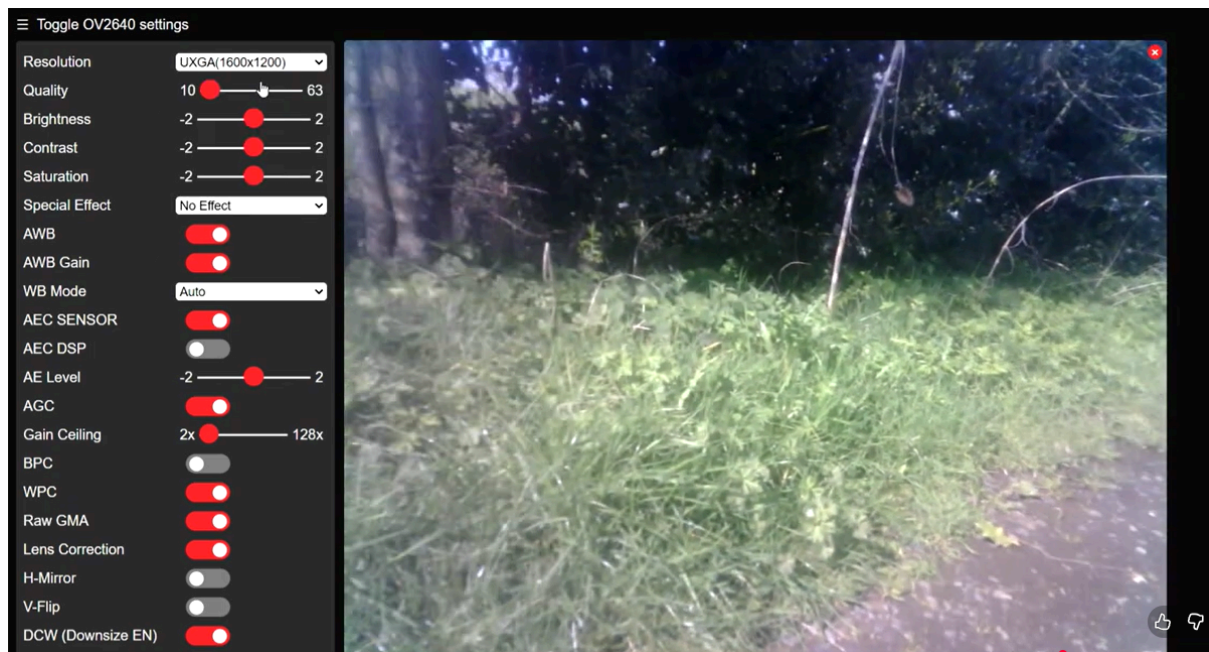
5. Lições Aprendidas

Dentre as lições aprendidas podemos destacar:

- Configuração das portas do ESP32 para Output, para acender os LED's
- Configuração da rede do microcontrolador para fazer um servidor web
- Criação de uma calculadora em complemento 2 e sincronizar com os LED's

6. Projeto

O projeto consiste em um sistema de segurança que simula o funcionamento de um porteiro eletrônico. A ideia central é utilizar um módulo com câmera — ESP32-CAM ou câmera do Raspberry Pi, conforme orientação do professor — integrado a um computador central, que recebe o stream de vídeo e exibe uma interface de monitoramento em tempo real



<https://www.youtube.com/watch?v=YG08SI1JbQw>

Um botão físico do kit funciona como acionador de alerta: ao identificar algo suspeito, o segurança o pressiona e o sistema envia uma notificação de emergência. A referência técnica principal para a implementação com ESP32-CAM é o tutorial, que demonstra a configuração do streaming via Wi-Fi. A versão com Raspberry Pi seguirá arquitetura equivalente, substituindo o módulo ESP32-CAM pela câmera dedicada da plataforma.